什么是事务

要么都成功，要么都失败（多条sql的执行）

事务的四大特性

原子性

一致性

隔离性

持久性

单条sql可以开启事务?

可以 begin,commit

spring的事务和mysql的事务有什么区别?

spring中的事务最终还是包装语句发给mysql进行事务控制，spring中只是对事务的逻辑做了一定的封装

mysql中事务

```
BEGIN;
INSERT into user (id,name,age) values(1,"wentai",18);
SAVEPOINT A;
INSERT into user (id,name,age) values(2,"tom",20);
ROLLBACK to A;
commit;
```

开启事务的方式有哪些

编程式：使用TransactionTemplate

这个里面有一些方法，打开源码看下

```
TransactionTemplate
    TransactionTemplate()
    TransactionTemplate(PlatformTransactionManager)
    TransactionTemplate(PlatformTransactionManager, TransactionDefinitior
    setTransactionManager(PlatformTransactionManager): void
    getTransactionManager(): PlatformTransactionManager
    afterPropertiesSet(): void
    execute(TransactionCallback<T>): T ↑TransactionOperations
    rollbackOnException(TransactionStatus, Throwable): void
    equals(Object): boolean ↑DefaultTransactionDefinition
    logger: Log = LogFactory.getLog(...)
    transactionManager: PlatformTransactionManager
```

声明式：使用Transactional注解

spring中的事务注解有哪些参数配置

1.propagation传播机制

2.isolation隔离级别

3.timeout超时时间，如果共用一个事务，只有之前的事务设置的timeout有效

4.rollbackFor回滚异常

5.noRollbackFor不会滚异常

事务传播机制

- REQUIRED(必须的)
  - 如果存在一个事务，则支持当前事务。如果没有事务则开启一个新的事务
- REQUIRES_NEW(需要新的)
  - 总是开启一个新的事务。如果一个事务已经存在，则将这个存在的事务挂起
- SUPPORTS
  - 支持当前事务，如果没有当前事务，就以非事务方法执行
- MANDATORY(强制性的)
  - 使用当前事务，如果没有当前事务，就抛出异常
- NOT_SUPPORTED
  - 总是非事务地执行，并挂起任何存在的事务
- NEVER
  - 总是非事务地执行，如果存在一个活动事务，则抛出异常
- NESTED(嵌套的);
  - 如果一个活动的事务存在，则运行在一个嵌套的事务中. 如果没有活动事务, 则按REQUIRED 属性执行。

spring中事务的隔离级别

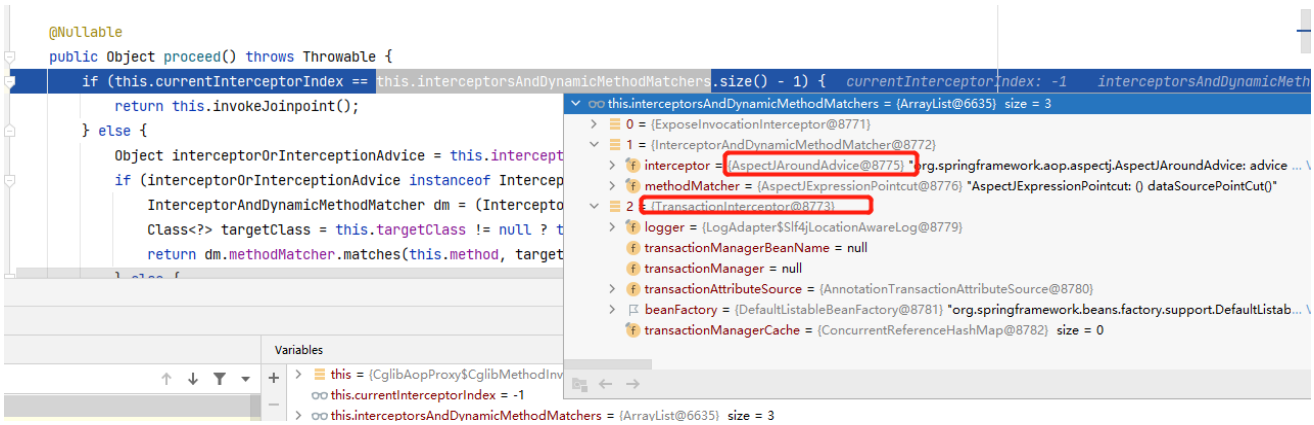
```
// 数据库默认的隔离级别
DEFAULT(-1),
// 读未提交 ru,会导致脏读
// 一事务对数据进行了增删改，但未提交，另一事务可以读取到未提交的数据。如果第一个事务这时候回滚了，那么第二个事务就读到了脏数据。
READ_UNCOMMITTED(1),
// 读已提交 rc 避免脏读，允许不可重复读和幻读
// 不可重复读：一个事务中发生了两次读操作，读到的结果不一致，针对其他事务有更新操作或者删除操作
// 幻读一个事务中发生了两次读操作，读到的结果不一致，针对其他事务有新增
READ_COMMITTED(2),
// 可重复读 rr 避免脏读，不可重复读，允许幻读
REPEATABLE_READ(4),
// 序列化
SERIALIZABLE(8);
// 从上到下，隔离级别越来越高，并发性能就越来越差
```

通过动态数据源的例子，来看事务

业务场景：要从从库中查出User对象，然后封装成UserPay对象插入到主库中

问题：为什么REQUIRES_NEW切换数据源成功了，而REQUIRED数据源缺切换不成功呢？

我们先来看看我们总共有多少的切面：我们总共有两个切面



那么那个切面先执行呢？我们看下自己定义切面的order的大小

```
ReflectiveMethodInvocation.class   DataSourceAspect.java   Ordered.class
                dynamicDataSource.clearDataSource();
                log.debug("clean datasource");
            }
        }

        @Override
        public int getOrder() {
            return 0;
        }
```

然后看下事务的order的大小AbstractPointcutAdvisor类中的order默认是2147483647，int的最大值，由于order值越小，则先执行，所以一般我们自定义的切面一定会优先于事务的切面执行

compiled .class file, bytecode version: 52.0 (Java 8)

```
public AbstractPointcutAdvisor() {
}

public void setOrder(int order) { this.order = order; }

public int getOrder() {
    if (this.order != null) {
        return this.order;
    } else {
        Advice advice = this.getAdvice();
        return advice instanceof Ordered ? ((Ordered)advice).getOrder() : 2147483647;
    }
}

public boolean isPerInstance() { return true; }
```

```
public Object invoke(MethodInvocation mi) throws Throwable {   mi: "ReflectiveMethodInvocation: public void com.gupao
    MethodInvocation oldInvocation = (MethodInvocation)invocation.get();   oldInvocation: null
    invocation.set(mi);   mi: "ReflectiveMethodInvocation: public void com.gupaoedu.service.impl.PayServiceImpl.payAm

    Object var3;
    try {
        var3 = mi.proceed();
    } finally {
        invocation.set(oldInvocation);
    }

    return var3;
}
```

然后执行我们自定义的切面方法

```java
tiveMethodInvocation.java  MapperTest.java  MethodInterceptor.java  AspectJAroundAdvice.java  AbstractAspectJAdvice.java
52      @Override
53      public boolean isAfterAdvice() { return false; }
56
57      @Override
58      protected boolean supportsProceedingJoinPoint() { return true; }
61
62      @Override
63      public Object invoke(MethodInvocation mi) throws Throwable {
64          if (!(mi instanceof ProxyMethodInvocation)) {
65              throw new IllegalStateException("MethodInvocation is not a Spring ProxyMethodInvocation: " + mi)
66          }
67          ProxyMethodInvocation pmi = (ProxyMethodInvocation) mi;
68          ProceedingJoinPoint pjp = lazyGetProceedingJoinPoint(pmi);
69          JoinPointMatch jpm = getJoinPointMatch(pmi);
70          return invokeAdviceMethod(pjp, jpm, returnValue: null, t: null);
71      }
72
```

Return the ProceedingJoinPoint for the current invocation, instantiating it lazily if it hasn't been bound to the thread already.

Params: rmi – the current Spring AOP ReflectiveMethodInvocation, which we'll use for attribute binding

然后执行我们的定义的切面的around方法

```java
veMethodInvocation.java  MapperTest.java  MethodInterceptor.java  AspectJAroundAdvice.java  AbstractAspectJAdvice.java  DataS
32
33          return invokeAdviceMethodWithGivenArgs(argBinding(jp, jpMatch, returnValue, t));
34      }
35
36      protected Object invokeAdviceMethodWithGivenArgs(Object[] args) throws Throwable {
37          Object[] actualArgs = args;
38          if (this.aspectJAdviceMethod.getParameterCount() == 0) {
39              actualArgs = null;
40          }
41          try {
42              ReflectionUtils.makeAccessible(this.aspectJAdviceMethod);
43              // TODO AopUtils.invokeJoinpointUsingReflection
44              return this.aspectJAdviceMethod.invoke(this.aspectInstanceFactory.getAspectInstance(), actualArgs);
45          }
46          catch (IllegalArgumentException ex) {
47              throw new AopInvocationException("Mismatch on arguments to advice method [" +
48                      this.aspectJAdviceMethod + "]; pointcut expression [" +
49                      this.pointcut.getPointcutExpression() + "]", ex);
50          }
51          catch (InvocationTargetException ex) {
52              throw ex.getTargetException();
```

然后开始设置数据源，第一次进来的时候，我们设置的主库，然后就进入到我们自己的方法，那就是在进入我们自己的方法之前，他设置了数据源，进入我们自定义的切面然后又开始递归调用

```java
lectiveMethodInvocation.java  AspectJAroundAdvice.java  AbstractAspectJAdvice.java  DataSourceAspect.java  ExposeInvocationInterc

    }

    @Around("dataSourcePointCut()")
    public Object around(ProceedingJoinPoint point) throws Throwable {
        MethodSignature signature = (MethodSignature) point.getSignature();
        Method method = signature.getMethod();

        DataSource ds = method.getAnnotation(DataSource.class);
        if (ds == null) {
            DynamicDataSource.setDataSource(DataSourceNames.MASTER);
            log.debug("set datasource is " + DataSourceNames.MASTER);
        } else {
            DynamicDataSource.setDataSource(ds.name());
            log.debug("set datasource is " + ds.name());
        }

        try {
            return point.proceed();
        } finally {
            DynamicDataSource.clearDataSource();
            log.debug("clean datasource");
        }
```

进入到TransactionInterceptor类中的invoke方法

```java
tiveMethodInvocation.java  TransactionInterceptor.java  AspectJAroundAdvice.java  AbstractAspectJAdvice.java

    @Override
    @Nullable
    public Object invoke(MethodInvocation invocation) throws Throwable {  invocation: "ReflectiveM
        // Work out the target class: may be {@code null}.
        // The TransactionAttributeSource should be passed the target class
        // as well as the method, which may be from an interface.
        Class<?> targetClass = (invocation.getThis() != null ? AopUtils.getTargetClass(invocation.g

        // Adapt to TransactionAspectSupport's invokeWithinTransaction...
        return invokeWithinTransaction(invocation.getMethod(), targetClass, invocation::proceed);
```

进入到开启事务代码中

```
Advice.java  DataSourceAspect.java  ExposeInvocationInterceptor.java  TransactionAspectSupport.java  PlatformTransactionManager.java
        final InvocationCallback invocation) throws Throwable {

    // If the transaction attribute is null, the method is non-transactional.
    TransactionAttributeSource tas = getTransactionAttributeSource();
    final TransactionAttribute txAttr = (tas != null ? tas.getTransactionAttribute(method, targetClass) : null);
    final PlatformTransactionManager tm = determineTransactionManager(txAttr);
    final String joinpointIdentification = methodIdentification(method, targetClass, txAttr);

    if (txAttr == null || !(tm instanceof CallbackPreferringPlatformTransactionManager)) {
        // Standard transaction demarcation with getTransaction and commit/rollback calls.
        TransactionInfo txInfo = createTransactionIfNecessary(tm, txAttr, joinpointIdentification);

        Object retVal;
        try {
            // This is an around advice: Invoke the next interceptor in the chain.
            // This will normally result in a target object being invoked.
            retVal = invocation.proceedWithInvocation();
        }
        catch (Throwable ex) {
            // target invocation exception
            completeTransactionAfterThrowing(txInfo, ex);
            throw ex;
        }
        finally {
```

```
pectJAdvice.java  DataSourceAspect.java  ExposeInvocationInterceptor.java  TransactionAspectSupport.java  PlatformTransactionManager.java  AbstractP
        @Nullable TransactionAttribute txAttr, final String joinpointIdentification) {

    // If no name specified, apply method identification as transaction name.
    if (txAttr != null && txAttr.getName() == null) {
        txAttr = new DelegatingTransactionAttribute(txAttr) {
            @Override
            public String getName() { return joinpointIdentification; }
        };
    }

    TransactionStatus status = null;
    if (txAttr != null) {
        if (tm != null) {
            status = tm.getTransaction(txAttr);
        }
        else {
            if (logger.isDebugEnabled()) {
                logger.debug( o: "Skipping transactional joinpoint [" + joinpointIdentification +
                        "] because no transaction manager has been configured");
            }
        }
    }
    return prepareTransactionInfo(tm, txAttr, joinpointIdentification, status);
}
```

第一次进来的时候为PROPAGATION_REQUIRED，肯定会开一个事务，开启新事物就是把newTransaction设置为true,表示开启新事务

```java
// No existing transaction found -> check propagation behavior to find out how to proceed.
if (definition.getPropagationBehavior() == TransactionDefinition.PROPAGATION_MANDATORY) {
    throw new IllegalTransactionStateException(
            "No existing transaction found for transaction marked with propagation 'mandatory'");
}
else if (definition.getPropagationBehavior() == TransactionDefinition.PROPAGATION_REQUIRED ||
        definition.getPropagationBehavior() == TransactionDefinition.PROPAGATION_REQUIRES_NEW ||
        definition.getPropagationBehavior() == TransactionDefinition.PROPAGATION_NESTED) {
    SuspendedResourcesHolder suspendedResources = suspend( transaction: null);   suspendedResources: null
    if (debugEnabled) {   debugEnabled: false
        logger.debug( o: "Creating new transaction with name [" + definition.getName() + "]: " + definition);   definition: "PROPA
    }
    try {
        boolean newSynchronization = (getTransactionSynchronization() != SYNCHRONIZATION_NEVER);
        DefaultTransactionStatus status = newTransactionStatus(
                definition, transaction, newTransaction: true, newSynchronization, debugEnabled = false, suspendedResources);
        doBegin(transaction, definition);
```

这个时候需要去拿到数据源，回去调用我们在DynamicDataSource类中的determineCurrentLookupKey方法得到数据源，第一次肯定拿的是first

```java
This implementation sets the isolation level but ignores the timeout.
@Override
protected void doBegin(Object transaction, TransactionDefinition definition) {   transaction: DataSourceTransactionManager$DataSou
    DataSourceTransactionObject txObject = (DataSourceTransactionObject) transaction;   transaction: DataSourceTransactionManager$
    Connection con = null;   con: null

    try {
        if (!txObject.hasConnectionHolder() ||
                txObject.getConnectionHolder().isSynchronizedWithTransaction()) {   txObject: DataSourceTransactionManager$DataSou
            Connection newCon = obtainDataSource().getConnection();
            if (logger.isDebugEnabled()) {
                logger.debug( o: "Acquired Connection [" + newCon + "] for JDBC transaction");
            }
            txObject.setConnectionHolder(new ConnectionHolder(newCon), newConnectionHolder: true);
        }

        txObject.getConnectionHolder().setSynchronizedWithTransaction(true);
        con = txObject.getConnectionHolder().getConnection();

        Integer previousIsolationLevel = DataSourceUtils.prepareConnectionForTransaction(con, definition);
        txObject.setPreviousIsolationLevel(previousIsolationLevel);

        // Switch to manual commit if necessary. This is very expensive in some JDBC drivers,
```

由于是多个事务之前嵌套的，所以一般我们都要去把他设置为不要自动提交，不然我们根本控制不了多个事务的嵌套

```java
DataSourceTransactionManager.java | DataSourceAspect.java | DataSource.java | AbstractRoutingDataSource.java | DynamicDataSource.java | PlatformTran
            txObject.setConnectionHolder(new ConnectionHolder(newCon), newConnectionHolder: true);
        }

        txObject.getConnectionHolder().setSynchronizedWithTransaction(true);
        con = txObject.getConnectionHolder().getConnection();

        Integer previousIsolationLevel = DataSourceUtils.prepareConnectionForTransaction(con, definition);  previousIsolatio
        txObject.setPreviousIsolationLevel(previousIsolationLevel);  previousIsolationLevel: null   txObject: DataSourceTra

        // Switch to manual commit if necessary. This is very expensive in some JDBC drivers,
        // so we don't want to do it unnecessarily (for example if we've explicitly
        // configured the connection pool to set it already).
        if (con.getAutoCommit()) {  con: "com.alibaba.druid.proxy.jdbc.ConnectionProxyImpl@723e3c17"
            txObject.setMustRestoreAutoCommit(true);
            if (logger.isDebugEnabled()) {
                logger.debug( o: "Switching JDBC Connection [" + con + "] to manual commit");
            }
            con.setAutoCommit(false);
        }

        prepareTransactionalConnection(con, definition);
        txObject.getConnectionHolder().setTransactionActive(true);

        int timeout = determineTimeout(definition);
```

这是一个回调函数，最开始进入TransactionInterceptor的时候传入的，是ReflectiveMethodInvocation中的proceed方法

```java
TransactionAspectSupport.java | DataSourceTransactionManager.java | DataSourceAspect.java | DataSource.java | AbstractRoutingDataSource.java
    // If the transaction attribute is null, the method is non-transactional.
    TransactionAttributeSource tas = getTransactionAttributeSource();  tas: AnnotationTransactionAttributeSource@8319
    final TransactionAttribute txAttr = (tas != null ? tas.getTransactionAttribute(method, targetClass) : null);  tas.
    final PlatformTransactionManager tm = determineTransactionManager(txAttr);  tm: DataSourceTransactionManager@8329
    final String joinpointIdentification = methodIdentification(method, targetClass, txAttr);  method: "public void c

    if (txAttr == null || !(tm instanceof CallbackPreferringPlatformTransactionManager)) {
        // Standard transaction demarcation with getTransaction and commit/rollback calls.
        TransactionInfo txInfo = createTransactionIfNecessary(tm, txAttr, joinpointIdentification);  txAttr: "PROPAGA

        Object retVal;
        try {
            // This is an around advice: Invoke the next interceptor in the chain.
            // This will normally result in a target object being invoked.
            retVal = invocation.proceedWithInvocation();  invocation: TransactionInterceptor$lambda@8327
        }
        catch (Throwable ex) {
            // target invocation exception
```

到目前位置，我们先切换了数据源，然后在开启了事务，终于进入到我们的目标方法，但是payService.findUser4RequireNew这个方法又添加了事务和数据源，并且是RequireNew的，此时的payService肯定是一个代理类，然后又进入切面，要切换到从数据库查询数据，然后开启事务，然后提交事务，然后在把数据插入到主数据库

```
ReflectiveMethodInvocation.java | PayServiceImpl.java | DataSourceTransactionManager.java | DataSourceAspect.java | DataSource.java | AbstractRoutingDataSource.java | Dy
```

```java
    public PayServiceImpl() { System.out.println("......PayServiceImpl......"); }

    @Transactional
    @DataSource(name = DataSourceNames.MASTER)
    public void payAndFindWithRequireNew(String userId) {   userId: "990"
        logger.info("准备执行findUser4RequireNew方法");
        String userName = userId+ "_Name";   userId: "990"
        User user = payService.findUser4RequireNew(userName);

        if (null == user)
        {
//          System.out.println("从从库中拿到的user为空");
            logger.info("从从库中拿到的user为空，返回");

        }else
        {
            //insert
            //update
            logger.info("从从库中拿到了user: " + user.toString());
            logger.info("开始在主库中插入userPay记录!");
            //threadLocal.get
            //FIXME 打印SQL连接语句
            UserPay userPay1 = new UserPay();
```

```java
        boolean newSynchronization = (getTransactionSynchronization() == SYNCHRONIZATION_ALWAYS);
        return prepareTransactionStatus(
                definition,  transaction: null,  newTransaction: false, newSynchronization, debugEnabled, suspendedResources);
    }

    if (definition.getPropagationBehavior() == TransactionDefinition.PROPAGATION_REQUIRES_NEW) {
        if (debugEnabled) {
            logger.debug( o: "Suspending current transaction, creating new transaction with name [" +   logger: LogAdapter$Slf4jLocat
                    definition.getName() + "]");
        }
        SuspendedResourcesHolder suspendedResources = suspend(transaction);   suspendedResources: AbstractPlatformTransactionManager$
        try {
            boolean newSynchronization = (getTransactionSynchronization() != SYNCHRONIZATION_NEVER);   newSynchronization: true
            DefaultTransactionStatus status = newTransactionStatus(   status: DefaultTransactionStatus@8407
                    definition, transaction,  newTransaction: true, newSynchronization, debugEnabled, suspendedResources);   newSynchro
            doBegin(transaction, definition);   definition: "PROPAGATION_REQUIRES_NEW,ISOLATION_DEFAULT"   transaction: DataSourceT
            prepareSynchronization(status, definition);
            return status;
        }
        catch (RuntimeException | Error beginEx) {
            resumeAfterBeginException(transaction, suspendedResources, beginEx);
```

提交里面里面那一层事务

```java
// Standard transaction demarcation with getTransaction and commit/rollback calls.
TransactionInfo txInfo = createTransactionIfNecessary(tm, txAttr, joinpointIdentification);

Object retVal;
try {
    // This is an around advice: Invoke the next interceptor in the chain.
    // This will normally result in a target object being invoked.
    retVal = invocation.proceedWithInvocation();
}
catch (Throwable ex) {
    // target invocation exception
    completeTransactionAfterThrowing(txInfo, ex);
    throw ex;
}
finally {
    cleanupTransactionInfo(txInfo);
}
commitTransactionAfterReturning(txInfo);
return retVal;
}

else {
```

只有新事务才能提交

```
ivemethodInvocation.java × TransactionInterceptor.java × TransactionAspectSupport.java × PlatformTransactionManager.java × AbstractPlatformTra
try {
    boolean unexpectedRollback = false;
    prepareForCommit(status);
    triggerBeforeCommit(status);
    triggerBeforeCompletion(status);
    beforeCompletionInvoked = true;

    if (status.hasSavepoint()) {
        if (status.isDebug()) {
            logger.debug( o: "Releasing transaction savepoint");
        }
        unexpectedRollback = status.isGlobalRollbackOnly();
        status.releaseHeldSavepoint();
    }
    else if (status.isNewTransaction()) {
        if (status.isDebug()) {
            logger.debug( o: "Initiating transaction commit");
        }
        unexpectedRollback = status.isGlobalRollbackOnly();
        doCommit(status);
    }
    else if (isFailEarlyOnGlobalRollbackOnly()) {
        unexpectedRollback = status.isGlobalRollbackOnly();
```

然后再提交外面那层事务，所以由此可以看出我们REQUIRES_NEW会重新拿到数据源，开启新事务，然后去查询数据

- 我们再看REQUIRED的情况为什么失败?

  因为他没有重新去获取数据源

- Nested和require的区别?
  - Nested的事务不会影响外层的事务，require的事务会影响外层的事务
- Nested和requires_new的区别

- Nested里面的事务回滚，不会影响外面的事务，requires_new里面的事务回滚也不会影响外面的事务
- Nested不能单独提交，如果外层的事务回滚，里面的跟着回滚
- requires_new可以单独提交，就算外面的事务回滚了，里面的也可以提交

spring事务的失效场景

1.非public方法 事务的实现原理是代理增强，非public不能进行代理增强，不能进行JDK或者CGLIB代理

2.调用本类的方法 （this代表了是本类对象，而不是从容器中获取的代理对象，所以事务失效）

3.抛出捕捉的非RuntimeException，如果想要捕捉所有异常（rollbackFor = {Exception.class}）

或者指定@Transactional(rollbackFor = {RuntimeException.class,ClassNotFoundException.class},propagation = Propagation.REQUIRED) 多个异常类

4.我们知道事务就是依赖于数据库的，所以数据库不支持肯定也是失效的！！比如myIsam

5.如果AOP使用了CGLIB代理，事务方法或者类不是public，无法被外部包访问到，或者是final无法继承